

Group members: Ashwin Varkey 23115420

Chukwudi Williams 23061170

Manik Malik 23169717

Assignment 11

Amdahl's model:

2a)

$$\begin{aligned} S(N) &= \frac{Ts + Tp}{Ts + \frac{Tp}{N} + Tc(N)} \\ &= \frac{T(1)}{Ts + \frac{Tp}{N} + Tc(N)} \\ &= 1 / \left(\left(\frac{Ts}{T(1)} \right) + \left(\frac{Tp}{N * T(1)} \right) + \left(\frac{Tc(N)}{T(1)} \right) \right) \\ &= \frac{1}{s + \frac{p}{N} + c} \end{aligned}$$

2b)

$$\begin{aligned} S(N, \sigma) &= \frac{1}{\frac{s}{\sigma} + \frac{p}{\sigma N} + c} \\ &= \frac{1}{2s + \frac{2p}{N} + c} \end{aligned}$$

According to Amdahl's law, the problem size is set constant and the number of the processors working on the problem is increased. From the above equation we see that, when N tends to infinity, the parallel part is neglected. Since, communication overhead is a small value, it can also be neglected. Thus we are only left with the serial part. From Amdahl's law, the upper limit is dependent on the serial part of the code. Thus we cannot agree with the statement "Scaling of the code can be improved by making it run slower" By the factor of 0.5, the model will scale down by 2. Timing of the serial part is the only prerequisite for better scalability.

2c) $s=0$ (no serial fraction) and $c(N)=\alpha \log N$

$$P(N) = \frac{1}{\left(\frac{p}{N}\right) + \alpha \log N}$$
$$P(N, \sigma) = \frac{1}{\left(\frac{p}{N * \sigma}\right) + \alpha \log N}$$

For large values of N , the logarithmic term $\alpha \log N$ dominates the denominator, the performance scales as: N tends to infinity

$$P(N, \sigma) \approx 1 / (\alpha \log N)$$

The performance $P(N, \sigma)$ depends inversely on $\alpha \log N$. As the number of workers N increases, the logarithmic term in the denominator decreases, leading to higher performance. The performance also depends on the slowdown factor σ , as it scales proportionally with it.

This is an expected result because in the absence of any serial fraction ($s = 0$), the performance improvement is mainly determined by the communication overhead represented by the logarithmic term $\alpha \log N$. The slower the communication overhead (represented by a smaller α), the better the scaling performance.

Here the upper bound is defined by the constant latency, which is expected. Even Though, the limit of N does not depend on σ , it plays a role in reaching the upper bound. When $\sigma < 1$, the scalability will be slower towards the upper bound.

1) Optimal energy to solution

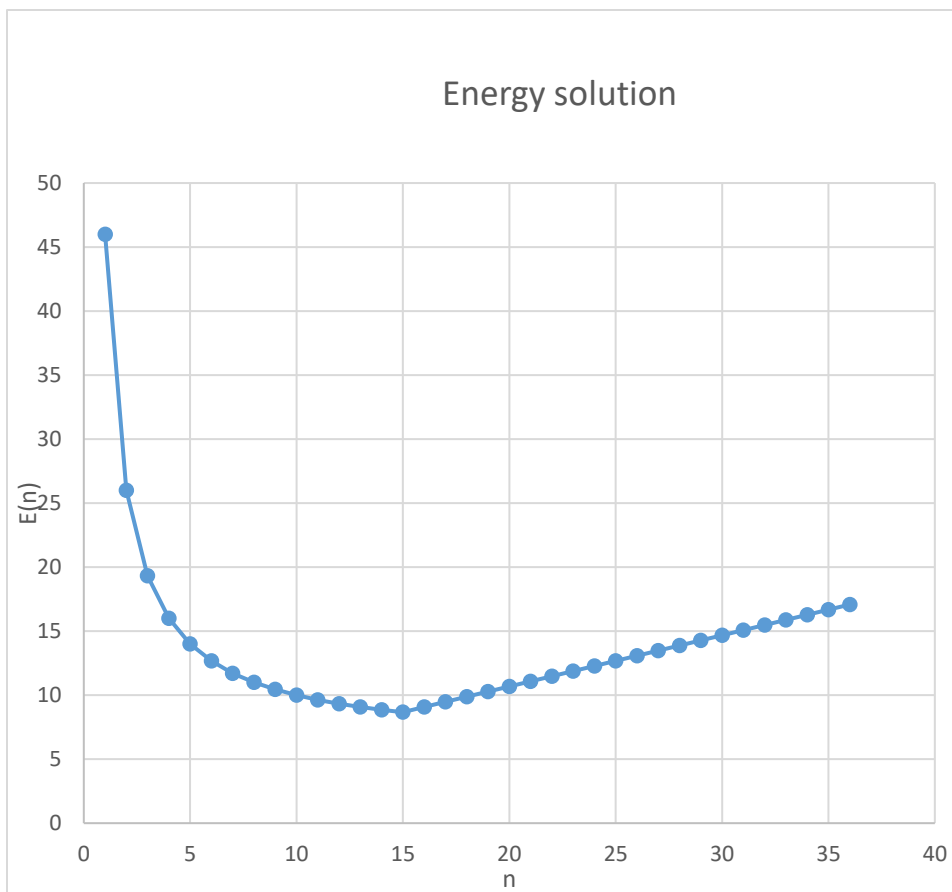
a)

$$E(n) = W(n)/T(n) \quad T(n) = 1/P(n)$$

$$P(n) = \min(nP(1), P_{\text{limit}}) \quad W(n) = W_0 + n \cdot W_d \quad n = 1, 2, \dots, 36$$

$$W_0 = 40W \quad W_d = 6W$$

$$P(1) = 1/s \quad P_{\text{limit}} = 15/s$$



For $n=15$, The $E(n)$ is minimal, if we increase the cores beyond this,

$E(n)$ shows an increasing trend which is not required.

n	E (n)
1	46
2	26
3	19.33
4	16
5	14
6	12.667
7	11.71428571
8	11
9	10.444
10	10
11	9.6363
12	9.333
13	9.0769
14	8.857
15	8.667
16	9.0667
17	9.4667
18	9.8677
19	10.2677
20	10.6667
21	11.0667
22	11.4667
23	11.8667
24	12.2667
25	12.6677
26	13.0667
27	13.4667
28	13.8667
29	14.2667
30	14.667
31	15.0667
32	15.4667
33	15.8667
34	16.2667
35	16.667
36	17.0667

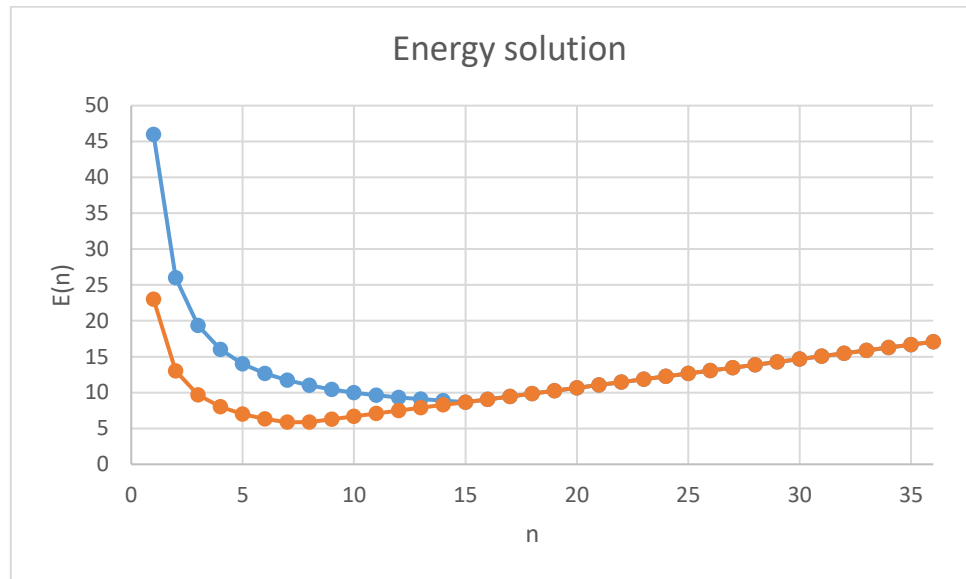
1b) $E(n) = W(n)/T(n)$ $T(n) = 1/P(n)$ SIMD vectorization

$P(n) = \min(nP(1), P_{\text{limit}})$ $W(n) = W_0 + n \cdot W_d$ $n = 1, 2, \dots, 36$

$W_0 = 40W$ $W_d = 6W$

$P(1) = 2/s$ $P_{\text{limit}} = 15/s$

n	E (n)
1	23
2	13
3	9.6667
4	8
5	7
6	6.3333
7	5.8571
8	5.8667
9	6.2667
10	6.6667
11	7.0667
12	7.4667
13	7.8667
14	8.2667
15	8.667
16	9.0667
17	9.4667
18	9.8677
19	10.2677
20	10.6667
21	11.0667
22	11.4667
23	11.8667
24	12.2667
25	12.6677
26	13.0667
27	13.4667
28	13.8667
29	14.2667
30	14.667
31	15.0667
32	15.4667
33	15.8667
34	16.2667
35	16.667
36	17.0667



$N(\min)=7$, $E(7)=5.8571$ J

From the plot, we can arrive to a conclusion that with SIMD vectorization it is possible to improve single core performance and reduce the number of cores required to reach solution with less energy to solution.

1c) $E(n) = W(n)/T(n)$ $T(n) = 1/P(n)$

$P(n) = \min(nP(1), P_{\text{limit}})$ $W(n) = W_0 + n \cdot W_d$ $n = 1, 2, \dots, 36$

$W_0 = 40W$ $W_d = 6W$

$P(1) = 2/s$ $P_{\text{limit}} = 25/s$

n	E (n)
1	23
2	13
3	9.6667
4	8
5	7
6	6.3333
7	5.8571
8	5.5
9	5.222
10	5
11	4.818
12	4.667
13	4.72
14	4.96
15	5.2
16	5.44
17	5.68
18	5.92
19	6.16
20	6.4
21	6.64
22	6.88
23	7.12
24	7.36
25	7.6
26	7.84
27	8.08
28	8.32
29	8.56
30	8.8
31	9.04
32	9.28
33	9.52
34	9.76
35	10
36	10.24

We can see that number of cores required to reach minimal solution increases, but the energy expenditure is less compared to previous case. n (min) is 12 with much lesser energy to solution.

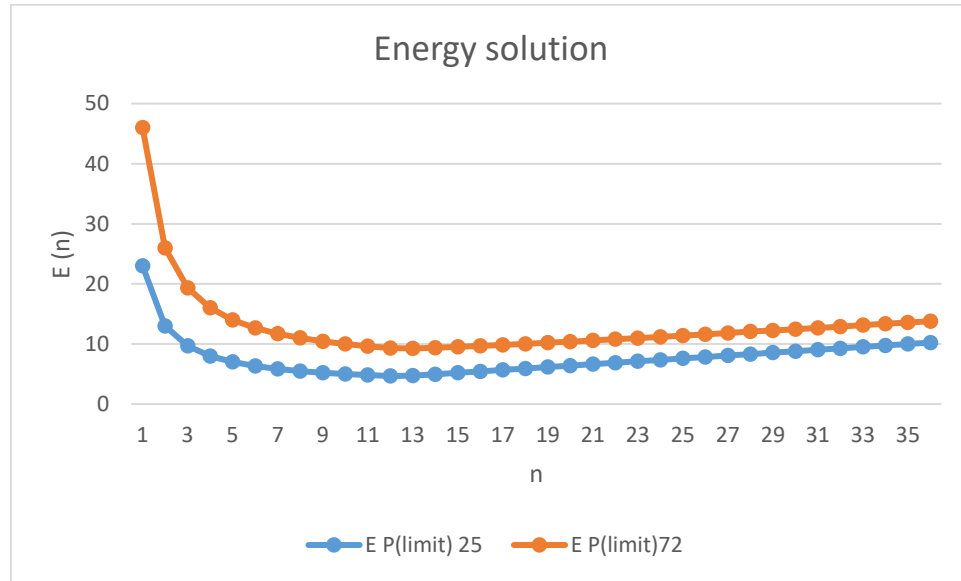
$$E(n) = W(n)/T(n) \quad T(n) = 1/P(n)$$

$$P(n) = \min(nP(1), P(\text{limit})) \quad W(n) = W_0 + n \cdot W_d \quad n = 1, 2, \dots, 36$$

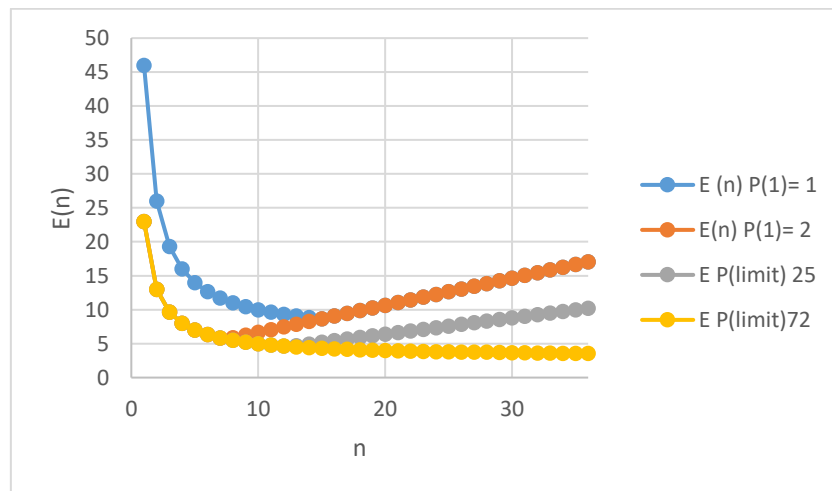
$$W_0 = 40W \quad W_d = 6W$$

$$P(1) = 2/s \quad P(\text{limit}) = 72/s$$

n	E (n)
1	23
2	13
3	9.6667
4	8
5	7
6	6.333
7	5.8571
8	5.5
9	5.222
10	5
11	4.818
12	4.667
13	4.5385
14	4.428
15	4.333
16	4.25
17	4.1765
18	4.1111
19	4.0526
20	4
21	3.9524
22	3.9091
23	3.8696
24	3.833
25	3.8
26	3.7692
27	3.7407
28	3.7143
29	3.6987
30	3.667
31	3.6542
32	3.625
33	3.6061
34	3.5882
35	3.5714
36	3.556



In this case, the energy expenditure does not reach a minimum value. With more cores, i.e., 36+ the energy expenditure keeps decreasing. Also, we could see that the energy expenditure for $P(\text{limit}) = 72/s$ value is lesser than the one captured for $P(\text{limit}) = 25/s$.



3.

		GB/s	c		
		0	1	2	3
m	0	73.27	73.534	52.994	53.126
	1	73.12	73.123	52.818	52.941
	2	52.913	52.89	52.89	73.39
	3	52.879	52.829	73.084	73.247

TEMPLATE: `srun --cpu-freq=2000000-2000000 numactl --membind=$m likwid-bench -t daxpy -w M$c:5000MB`

`srun --cpu-freq=2400000-2400000 numactl --membind=0 likwid-bench -t daxpy -w M0:5000MB`
`srun --cpu-freq=2400000-2400000 numactl --membind=0 likwid-bench -t daxpy -w M1:5000MB`
`srun --cpu-freq=2400000-2400000 numactl --membind=0 likwid-bench -t daxpy -w M2:5000MB`
`srun --cpu-freq=2400000-2400000 numactl --membind=0 likwid-bench -t daxpy -w M3:5000MB`

`srun --cpu-freq=2400000-2400000 numactl --membind=1 likwid-bench -t daxpy -w M0:5000MB`
`srun --cpu-freq=2400000-2400000 numactl --membind=1 likwid-bench -t daxpy -w M1:5000MB`
`srun --cpu-freq=2400000-2400000 numactl --membind=1 likwid-bench -t daxpy -w M2:5000MB`
`srun --cpu-freq=2400000-2400000 numactl --membind=1 likwid-bench -t daxpy -w M3:5000MB`

`srun --cpu-freq=2400000-2400000 numactl --membind=2 likwid-bench -t daxpy -w M0:5000MB`
`srun --cpu-freq=2400000-2400000 numactl --membind=2 likwid-bench -t daxpy -w M1:5000MB`
`srun --cpu-freq=2400000-2400000 numactl --membind=2 likwid-bench -t daxpy -w M2:5000MB`
`srun --cpu-freq=2400000-2400000 numactl --membind=2 likwid-bench -t daxpy -w M3:5000MB`

`srun --cpu-freq=2400000-2400000 numactl --membind=3 likwid-bench -t daxpy -w M0:5000MB`
`srun --cpu-freq=2400000-2400000 numactl --membind=3 likwid-bench -t daxpy -w M1:5000MB`
`srun --cpu-freq=2400000-2400000 numactl --membind=3 likwid-bench -t daxpy -w M2:5000MB`
`srun --cpu-freq=2400000-2400000 numactl --membind=3 likwid-bench -t daxpy -w M3:5000MB`